

# PyTorch Profiler 性能分析

## 目 录

|                                   |   |
|-----------------------------------|---|
| 1 引言：优化前先定位瓶颈 .....               | 2 |
| 2 PyTorch Profiler 工作流 .....      | 2 |
| 3 Chrome Trace 格式与 Perfetto ..... | 3 |
| 3.1 Chrome Trace 格式 .....         | 3 |
| 3.2 Perfetto 可视化 .....            | 3 |
| 4 自顶向下分析方法 .....                  | 3 |
| 5 外部 Bubble：算子间空隙 .....           | 4 |
| 5.1 Python 开销 .....               | 4 |
| 5.2 GPU-CPU 同步 .....              | 4 |
| 5.3 小算子过多 .....                   | 4 |
| 5.4 频繁内存分配 .....                  | 4 |
| 6 内部 Bubble：算子内停顿 .....           | 4 |
| 7 Prefill 与 Decode 的性能特征 .....    | 5 |
| 8 实战：读一张 Timeline 示意图 .....       | 5 |
| 9 框架中的性能指标 .....                  | 6 |
| 10 性能分析实践 .....                   | 6 |
| 11 本章你将学会 .....                   | 7 |
| 12 要点速查 .....                     | 7 |
| 13 小结 .....                       | 7 |

# 1 引言：优化前先定位瓶颈

LLM 推理是一个异构应用：CPU 负责调度、Python 逻辑和数据准备，GPU 负责密集计算。两者通过 PCIe 总线协作，任何一方慢了都会拖累整体性能。在动手优化之前，我们必须先回答一个关键问题：瓶颈到底在 CPU 还是 GPU？

如果不做性能分析就盲目优化，可能花了一天时间把 GPU kernel 加快了 3 倍，结果发现真正的瓶颈是 CPU 上的 Python 循环，端到端只快了 5%。**PyTorch Profiler**（PyTorch 性能分析器）是定位这类瓶颈的标准工具，它记录 CPU 端函数调用和 GPU 端 kernel 执行的时间线，帮助我们精确定位性能浪费发生在哪里。

**直觉** 想象一条工厂流水线，前半段是人工操作（CPU），后半段是机器加工（GPU）。如果你只盯着机器说“机器转得不够快，换台更好的”，但实际瓶颈是工人手速太慢导致机器经常空转，那换再好的机器也没用。Profiler 就是给你一双能看到整条流水线的眼睛，让你一眼看出是工人慢了还是机器慢了。

## 2 PyTorch Profiler workflow

使用 PyTorch Profiler 分四步：

1. **包裹分析区域**：在代码中用 `with torch.profiler.profile(...)` 包裹需要分析的代码段
2. **指定事件类型和输出路径**：配置要记录的事件（CPU 算子、GPU kernel、内存分配等）和 trace 文件输出路径
3. **运行程序**：执行推理流程，Profiler 自动收集时间线数据
4. **可视化分析**：用 perfetto 或 `chrome://tracing` 打开 trace 文件

```
import torch

with torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
    ],
    record_shapes=True,
    on_trace_ready=torch.profiler.tensorboard_trace_handler("./trace"),
) as prof:
    model.generate(input_ids, max_new_tokens=128)

prof.export_chrome_trace("./trace.json")
```

逐行说明：activities 指定同时记录 CPU 和 CUDA 事件。record\_shapes 记录张量形状，便于定位不同 shape 的 kernel。on\_trace\_ready 在 trace 收集完成后自动写入指定目录。最后调用 export\_chrome\_trace 导出标准 Chrome Trace JSON 文件。

分析区域应包含足够多的迭代步骤（如 10 步 decode）以获得稳定的时间统计，但不要包含模型加载和预热阶段，否则会引入与推理无关的开销。

## 3 Chrome Trace 格式与 Perfetto

### 3.1 Chrome Trace 格式

Profiler 导出的 trace 文件是 JSONL (JSON Lines) 格式，每一行是一个事件对象。事件分为两类：

1. **CPU 事件**：Python 函数调用、PyTorch 算子调度、内存分配等
2. **GPU 事件**：CUDA kernel 的执行、CUDA Stream 同步等

每个事件记录名称、起止时间戳、所在线程或 Stream、持续时间等。CPU 事件和 GPU 事件通过 **correlation id** 关联：CPU 端发起一个 kernel 调用时分配一个 id，GPU 端执行该 kernel 时带上同一个 id，这样可以在 timeline 中看到某个 CPU 操作对应哪个 GPU kernel。

### 3.2 Perfetto 可视化

**Perfetto** 是 Google 开发的通用 trace 可视化工具，在线访问 [ui.perfetto.dev](https://ui.perfetto.dev)。相比 <chrome://tracing>，Perfetto 支持更大的 trace 文件、更流畅的缩放和 SQL 查询。

打开 trace 文件后，你会看到多个 **Track** (轨道)：CPU 线程轨道显示 Python 函数和算子调度，CUDA Stream 轨道显示 GPU kernel 执行。时间轴从左到右，每个色块代表一个事件，宽度即持续时间。

**直觉** Timeline 就像一张工厂的监控录像带，你能看到每个工人（CPU 线程）和每台机器（GPU Stream）在每一时刻在做什么。色块之间的空白就是空闲，色块的颜色可以区分不同类型的操作。

## 4 自顶向下分析方法

拿到 trace 后，按以下步骤分析：

1. **看全局**：先缩放到整个推理过程，观察 CPU 轨道和 GPU 轨道的繁忙程度。如果 CPU 忙而 GPU 空闲，瓶颈在 CPU；如果 GPU 忙而 CPU 空闲，瓶颈在 GPU；如果两者都忙但有大量空隙，说明存在同步等待
2. **定位热点**：在 GPU 轨道中找最宽的色块（执行时间最长的 kernel），这是计算热点
3. **分析空隙**：在 GPU 轨道中找连续的空白区域，这是 GPU 空闲的 bubble。追溯到对应的 CPU 时间点，看 CPU 在做什么
4. **检查同步**：寻找 CPU 上的 `synchronize` 或 `wait` 调用，这些是导致 GPU 空闲的常见原因

| 现象           | 可能原因       | 消除方法                            |
|--------------|------------|---------------------------------|
| GPU 空闲多      | Python 开销大 | 减少 Python 循环，合并操作               |
| GPU 空闲多      | GPU-CPU 同步 | 消除不必要的 <code>synchronize</code> |
| GPU 空闲多      | 小算子过多      | 算子融合，减少 kernel 发射               |
| GPU 空闲多      | 频繁内存分配     | 预分配缓冲区复用                        |
| GPU kernel 慢 | 访存瓶颈       | 分块计算，提高数据复用                     |

## 5 外部 Bubble: 算子间空隙

**External Bubble**（外部气泡）指 GPU kernel 之间的空闲时段，GPU 没有计算任务。常见来源有四类：

### 5.1 Python 开销

PyTorch 的 Python 前端每次调用一个算子，都要经过 Python 解释器、张量分发、autograd 记录等步骤。这些 CPU 操作在 kernel 发射之前完成，GPU 在此期间空闲。当算子很小（执行几微秒）而 Python 开销几毫秒时，GPU 等待比例极高。

消除方法：用 **CUDA Graph**（CUDA 图）把一组 kernel 录制为一个图，之后一次 launch 执行整个图，跳过逐个 kernel 的 Python 调度。

### 5.2 GPU-CPU 同步

某些操作需要 CPU 读取 GPU 的结果（如 `tensor.item()`、`tensor.cpu()`、`torch.cuda.synchronize()`），这会强制 CPU 等待 GPU 完成所有排队的 kernel。在等待期间，CPU 无法发射新的 kernel，GPU 在当前 kernel 完成后也因为没有新任务而空闲。

消除方法：避免在推理热路径中读取 GPU 结果，推迟到所有生成完成后再汇总。

### 5.3 小算子过多

多个小算子各自执行时间短但发射开销固定，累积起来形成大量间隙。消除方法：将多个小算子合并为一个大 kernel（算子融合），减少 kernel 发射次数。

### 5.4 频繁内存分配

PyTorch 的 `torch.empty()` 调用 CUDA 内存分配器，可能触发 `cudaMalloc` 或缓存查找。频繁分配释放导致碎片和延迟。消除方法：预分配缓冲区，在迭代间复用。

## 6 内部 Bubble: 算子内停顿

**Internal Bubble**（内部气泡）指 kernel 执行期间的内部停顿，GPU 虽然在“运行”该 kernel 但部分计算单元空闲。来源是访存等待：计算单元需要数据但数据还没从全局显存读到位。

**直觉** 想象一个厨师（计算单元）在做菜。食材（数据）放在仓库（全局显存）里，每次要用都要去仓库取。如果仓库太远或取货太慢，厨师就会站在灶台前等食材，灶火空烧。这就是内部 bubble：kernel 在执行但效率低下，因为计算单元在等数据。

消除方法：

1. **分块计算**：把数据分成小块放入片上 SRAM，减少对全局显存的访问
2. **合并访存**：把多次小读取合并为一次大读取，提高带宽利用率
3. **提高复用率**：同一块数据在 SRAM 中被多次使用后再换出，减少重复读取

内部 *bubble* 在 *timeline* 上不容易直接看到，因为它表现为 *kernel* 执行时间长而非空白。需要通过 *kernel* 的理论计算时间与实测时间的对比来判断：如果实测远大于理论，说明存在内部 *bubble*。

## 7 Prefill 与 Decode 的性能特征

同一套推理代码在 prefill 和 decode 两种阶段下的性能特征截然不同：

1. **Prefill**: 输入是长度为  $q_{len}$  的 prompt，注意力是矩阵乘矩阵 ( $q_{len} \times L$ )，GEMM 计算量大，GPU 并行度高，Tensor Core 利用充分。瓶颈通常在计算
2. **Decode**: 输入是 1 个 token，注意力退化为矩阵乘向量，算术强度极低（每读一个权重只做极少计算）。瓶颈通常在访存和算子发射开销

| 特征        | Prefill        | Decode         |
|-----------|----------------|----------------|
| $q_{len}$ | $> 1$          | $= 1$          |
| 矩阵类型      | 矩阵 $\times$ 矩阵 | 矩阵 $\times$ 向量 |
| 算术强度      | 高              | 低              |
| 瓶颈        | 计算             | 访存 + 发射开销      |
| 最优 tile   | 大 $M$ 维        | 减少权重重复读取       |

Profiling 时应分别采集 prefill 和 decode 的 trace，单独分析各自瓶颈。同一 kernel 的最优 tile 参数在两种阶段下不同。

## 8 实战：读一张 Timeline 示意图

**例** 考虑以下 timeline（文字描述）：

```
CPU: [launch kernel_A] [ .item() sync ] [launch kernel_B] [launch kernel_C]
GPU:      [ kernel_A ] [ idle ] [ kernel_B ] [ kernel_C ]
           ^^^^^^^
           外部 bubble
```

分析：

第一步，CPU 发射 kernel\_A，GPU 开始执行。第二步，CPU 调用 .item() 读取 kernel\_A 的输出，这触发 GPU-CPU 同步，CPU 阻塞等待 kernel\_A 完成。第三步，kernel\_A 完成后 CPU 解除阻塞，但此时 GPU 没有排队任务，出现空闲（外部 bubble）。第四步，CPU 发射 kernel\_B，GPU 空闲结束开始执行。

瓶颈在 CPU: .item() 同步导致 GPU 空等。消除方法是将 .item() 推迟到所有 kernel 执行完毕后，或改用异步方式（如延迟读取、torch.cuda.Event）避免同步。

进一步看 kernel\_B: 假设它的实测执行时间是 2 ms，但理论计算量只需 0.5 ms。多出的 1.5 ms 是内部 bubble，来自访存等待。消除方法是增大 tile 或改进访存合并。

## 9 框架中的性能指标

Lab5 的实验框架提供了 `measure_operation` 上下文管理器和 `OperationMetrics` 数据类，用于在代码中嵌入细粒度性能测量。

```
@dataclass
class OperationMetrics:
    latency_s: float           # 操作总延迟 (秒)
    peak_allocated_bytes: int  # 峰值已分配显存
    peak_reserved_bytes: int  # 峰值预留显存

@contextmanager
def measure_operation(
    name: str,
    synchronize_metrics: bool = True,
):
    if synchronize_metrics:
        torch.cuda.synchronize()
    t0 = time.perf_counter()
    reset_peak_memory()
    yield
    if synchronize_metrics:
        torch.cuda.synchronize()
    t1 = time.perf_counter()
    print(f"{name}: {t1-t0:.4f}s")
```

逐行说明：`OperationMetrics` 记录三个核心指标：`latency_s` 是操作耗时，`peak_allocated_bytes` 是该操作期间峰值已分配显存，`peak_reserved_bytes` 是 PyTorch 缓存分配器预留的峰值显存。`measure_operation` 是一个 context manager：进入时可选同步 GPU 并重置显存计数器，退出时再次同步并记录时间。`synchronize_metrics` 控制是否调用 `torch.cuda.synchronize()`，设为 `True` 时测量的是包含 GPU 完成时间的真实延迟，设为 `False` 时只测量 CPU 端的发射时间。

`synchronize_metrics` 的选择很关键：测量 *kernel* 真实执行时间必须同步 (`True`)；但如果要模拟连续推理的流水效果、观察 CPU 是否成为瓶颈，则不同步 (`False`) 更有意义。

## 10 性能分析实践

优化不是一蹴而就的，推荐遵循以下 workflow：

1. **建立基线**：在不做任何优化的情况下运行端到端评测，记录吞吐量、TTFT、TPOT 和峰值显存。这是后续优化的对照基准
2. **验证正确性**：每次修改后先用小规模测试验证输出正确，确保优化没有破坏精度。错误的优化毫无意义
3. **Microbenchmark**：对单个算子或单个操作做独立性能测试 (microbenchmark)，排除其他因素的干扰，精确测量优化前后差异

4. **结合 Profiler 调优**：用 PyTorch Profiler 定位热点和 bubble，有针对性地优化。每做一轮优化后重新 profiling，验证瓶颈是否消除
5. **回到端到端评测**：确认单点优化有效后，回到端到端评测验证整体收益。有时单点优化因其他瓶颈存在而看不到端到端收益，需要继续处理下一个瓶颈

**直觉** 性能优化就像修一条拥堵的马路。先量整体流量（基线），找到最堵的路口（Profiler 定位瓶颈），拓宽它（优化），再量整体流量看是否改善。如果某个路口拓宽了但整体还是堵，说明下一个瓶颈转移到了别处，继续找下一个。逐个消除瓶颈，直到整体流畅。

## 11 本章你将学会

1. 用 `torch.profiler.profile` 包裹推理代码并导出 Chrome Trace 文件
2. 用 Perfetto 或 `chrome://tracing` 可视化 trace 并识别 CPU/GPU 瓶颈
3. 自顶向下分析 timeline：看全局、定位热点、分析空隙、检查同步
4. 区分外部 bubble（算子间空隙）的四类来源和对应消除方法
5. 区分内部 bubble（算子内停顿）的访存原因和消除方法
6. 解释 prefill 和 decode 阶段截然不同的性能特征和最优策略
7. 使用框架的 `measure_operation` 和 `OperationMetrics` 做细粒度性能测量
8. 遵循“基线 → 正确性 → microbenchmark → profiler 调优 → 端到端”的优化 workflow

## 12 要点速查

| 要点                               | 说明                                                                                            |
|----------------------------------|-----------------------------------------------------------------------------------------------|
| Profiler  workflow               | 包裹区域 → 指定事件 → 运行 → 可视化                                                                        |
| Chrome Trace                     | JSONL 格式，CPU 事件 + GPU 事件，correlation id 关联                                                    |
| Perfetto                         | <code>ui.perfetto.dev</code> ，支持大文件和 SQL 查询                                                   |
| 自顶向下分析                           | 看全局 → 定位热点 → 分析空隙 → 检查同步                                                                      |
| 外部 bubble                        | 算子间空隙，来自 Python 开销 / 同步 / 小算子 / 内存分配                                                          |
| 外部 bubble 消除                     | CUDA Graph / 去同步 / 算子融合 / 缓冲区复用                                                               |
| 内部 bubble                        | 算子内停顿，来自访存等待                                                                                  |
| 内部 bubble 消除                     | 分块计算 / 合并访存 / 提高复用率                                                                           |
| Prefill 瓶颈                       | 计算密集，大 $M$ tile                                                                               |
| Decode 瓶颈                        | 访存密集，算子发射开销大                                                                                  |
| OperationMetrics                 | <code>latency_s</code> , <code>peak_allocated_bytes</code> , <code>peak_reserved_bytes</code> |
| <code>synchronize_metrics</code> | True 测真实延迟，False 测 CPU 发射时间                                                                   |
| 优化 workflow                      | 基线 → 正确性 → microbenchmark → profiler → 端到端                                                    |

## 13 小结

PyTorch Profiler 是 LLM 推理优化的导航工具。它通过 Chrome Trace 记录 CPU 和 GPU 的完整执行时间线，帮助我们在动手优化之前精确定位瓶颈所在。自顶向下的分析方法从全局



繁忙度入手，逐步深入到热点 kernel 和空隙分析，最终区分出外部 bubble 和内部 bubble 两类性能浪费。

外部 bubble 来自算子间的空闲，根源是 Python 开销、GPU-CPU 同步、小算子过多和频繁内存分配，可通过 CUDA Graph、去同步、算子融合和缓冲区复用消除。内部 bubble 来自算子内的访存等待，可通过分块计算和合并访存缓解。Prefill 和 decode 两种阶段性能特征截然不同，需要分别 profiling 和优化。框架的 measure\_operation 提供了细粒度的延迟和显存测量能力。结合“基线 → 正确性 → microbenchmark → profiler 调优 → 端到端”的工作流，我们可以逐个消除瓶颈，系统性地提升推理性能。

讲义基于 HPC Lab5 实验指导编写